

Task 1.**1.1. PART A:**

1. Superkeys:

{EmpID}

{EmpID, Name}

{EmpID, Salary}

{EmpID, Department}

{EmpID, Phone}

{EmpID, Name, Salary}

2. Candidate keys:

{EmpID}, {SSN}, {Email}

3. {EmpID}, because it uniquely identifies each employee and is specifically designed for identification purposes. For example, Email may be change over time, but EmpID is expected to remain stable. It is also simpler and more appropriate than using sensitive information such as SSN.

4. Yes, two employees can have the same phone number. Although all phone numbers in the sample data are different, there is no business rule stating that Phone must be unique. Therefore, based only on the given data, we cannot assume that Phone uniquely identifies an employee.

PART B:

1. the minimum attributes for

Primary Key: {StudentID, CourseCode, Section, Semester, Year}

2. **StudentID** is necessary to identify which student is registered.

CourseCode is necessary because a student can register for different courses.

Section is necessary because the same course can have different sections.

Semester is necessary because a student can take the same course in different semesters.

Year is necessary to distinguish registrations in the same semester across different years, for example Fall 2026 and Fall 2027.

3. There are no additional candidate keys based on the given business rules.

1.2. 1. Student.AdvisorID → Professor.ProfID

AdvisorID identifies the professor who advises the student.

2. Course.DepartmentCode → Department.DeptCode

DepartmentCode identifies the department that offers the course.

3. Department.ChairID → Professor.ProfID

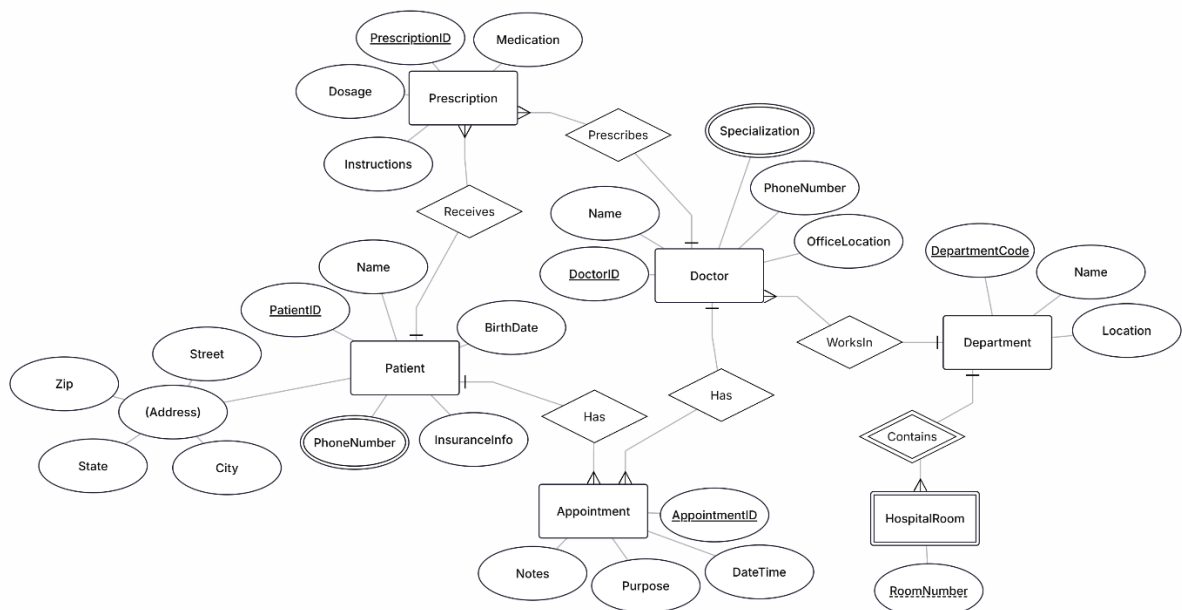
ChairID identifies the professor who is the chair of the department.

4. Enrollment.StudentID → Student.StudentID

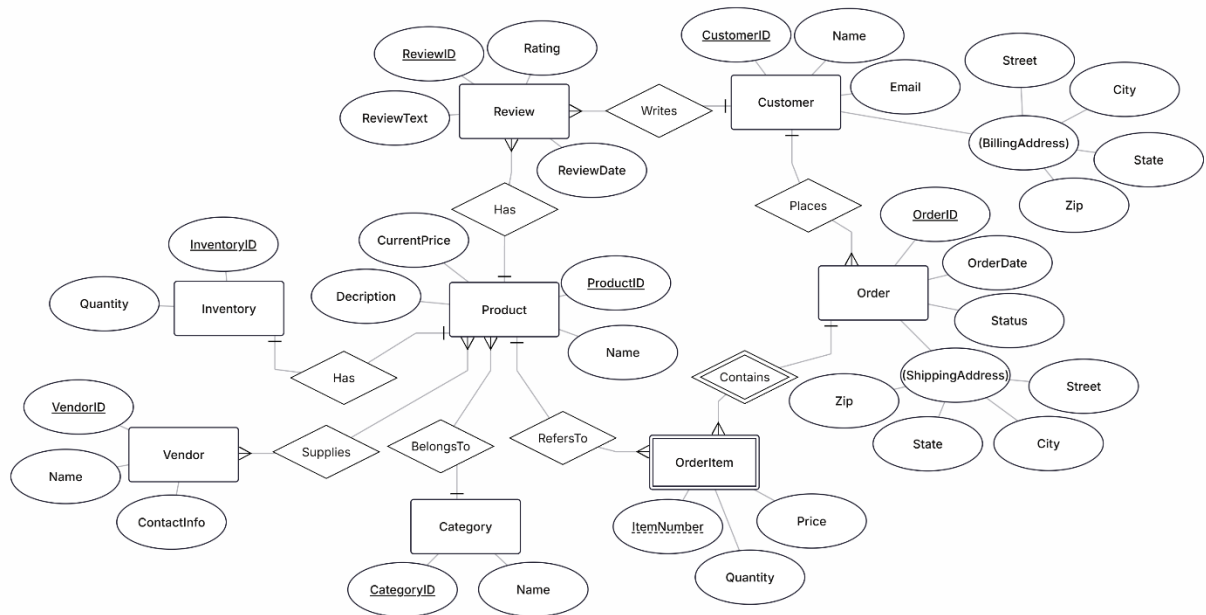
StudentID identifies the student who is enrolled.

5. Enrollment.CourseID → Course.CourseID

CourseID identifies the course in which the student is enrolled.

Task 2.**2.1. Hospital Management System**

2.2. E-commerce Platform



2) **Weak entity:** OrderItem is a weak entity because it cannot be uniquely identified independently of an Order. ItemNumber is only partially unique, and the complete identification of an OrderItem requires OrderID together with ItemNumber.

3) **Many-to-many relationship:** Orders and Products have a many-to-many relationship. An order can contain multiple products, and a product can appear in multiple orders. This relationship is resolved through the OrderItem entity, which stores the attributes Quantity and Price at the time of the order.

Task 4.

4.1. 1) The functional dependencies are:

StudentID $\rightarrow$ StudentName, StudentMajor

ProjectID $\rightarrow$ ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate

SupervisorID $\rightarrow$ SupervisorName, SupervisorDept

(StudentID, ProjectID) $\rightarrow$ Role, HoursWorked

2) Problems

Redundancy:

Student information is repeated for each project, while project and supervisor information is repeated for each participating student.

Update anomaly:

If supervisor information changes, it must be updated in every related row, which may cause inconsistencies.

Insert anomaly:

A new project cannot be stored until at least one student is assigned to it.

Delete anomaly:

Deleting the last student from a project may also remove important project and supervisor information.

3) Apply 1NF

The table is already in 1NF because every attribute contains a single atomic value and there are no repeating groups or multi-valued attributes.

4) Apply 2NF

The primary key is: (StudentID, ProjectID)

The partial dependencies are:

StudentID $\rightarrow$ StudentName, StudentMajor

ProjectID $\rightarrow$ ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, StartDate, EndDate

Role and HoursWorked depend on the complete composite key:

(StudentID, ProjectID) $\rightarrow$ Role, HoursWorked

The 2NF decomposition is:

STUDENT(StudentID PK, StudentName, StudentMajor)	PROJECT(ProjectID PK, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, StartDate, EndDate)	STUDENT_PROJECT(StudentID PK, FK, ProjectID PK, FK, Role, HoursWorked)
--	---	---

5) Apply 3NF

The transitive dependency is:

ProjectID $\rightarrow$ SupervisorID

SupervisorID $\rightarrow$ SupervisorName, SupervisorDept

Therefore:

ProjectID $\rightarrow$ SupervisorID $\rightarrow$ SupervisorName, SupervisorDept

The final 3NF decomposition is:

STUDENT(StudentID PK, StudentName, StudentMajor)	SUPERVISOR(SupervisorID PK, SupervisorName, SupervisorDept)	PROJECT(ProjectID PK, ProjectTitle, ProjectType, SupervisorID FK, StartDate, EndDate)	STUDENT_PROJECT(StudentID PK, FK, ProjectID PK, FK, Role, HoursWorked)
--	---	---	--

4.2. Advanced Normalization

1) The primary key is: (StudentID, CourseID, TimeSlot, Room)

StudentID identifies the student, while CourseID, TimeSlot, and Room identify the particular course section in which the student is enrolled.

2) The functional dependencies are:

- StudentID $\rightarrow$ StudentMajor
- CourseID $\rightarrow$ CourseName
- InstructorID $\rightarrow$ InstructorName
- Room $\rightarrow$ Building
- (CourseID, TimeSlot, Room) $\rightarrow$ InstructorID
- (StudentID, CourseID, TimeSlot, Room)
 $\rightarrow$ StudentMajor, CourseName, InstructorID, InstructorName, Building

3) The table is **not in BCNF**.

BCNF requires that the determinant of every non-trivial functional dependency be a superkey.

However, the following dependencies violate this requirement:

- $\text{StudentID} \rightarrow \text{StudentMajor}$
- $\text{CourseID} \rightarrow \text{CourseName}$
- $\text{InstructorID} \rightarrow \text{InstructorName}$
- $\text{Room} \rightarrow \text{Building}$

StudentID, CourseID, InstructorID, and Room are not superkeys of the original CourseSchedule relation. Therefore, the relation does not satisfy BCNF.

4) BCNF Decomposition

STUDENT(StudentID PK, StudentMajor)	COURSE(CourseID PK, CourseName)	INSTRUCTOR(InstructorID PK, InstructorName)	ROOM(Room PK, Building)
COURSE_SECTION(CourseID PK, FK, TimeSlot PK, Room PK, FK, InstructorID FK)	ENROLLMENT(StudentID PK, FK, CourseID PK, FK, TimeSlot PK, Room PK, FK)		

Foreign key relationships are:

- $\text{ENROLLMENT.StudentID} \rightarrow \text{STUDENT.StudentID}$
- $\text{ENROLLMENT}(\text{CourseID}, \text{TimeSlot}, \text{Room}) \rightarrow \text{COURSE_SECTION}(\text{CourseID}, \text{TimeSlot}, \text{Room})$
- $\text{COURSE_SECTION.CourseID} \rightarrow \text{COURSE.CourseID}$
- $\text{COURSE_SECTION.InstructorID} \rightarrow \text{INSTRUCTOR.InstructorID}$
- $\text{COURSE_SECTION.Room} \rightarrow \text{ROOM.Room}$

The composite key of COURSE_SECTION is:

- $(\text{CourseID}, \text{TimeSlot}, \text{Room})$

The composite key of ENROLLMENT is:

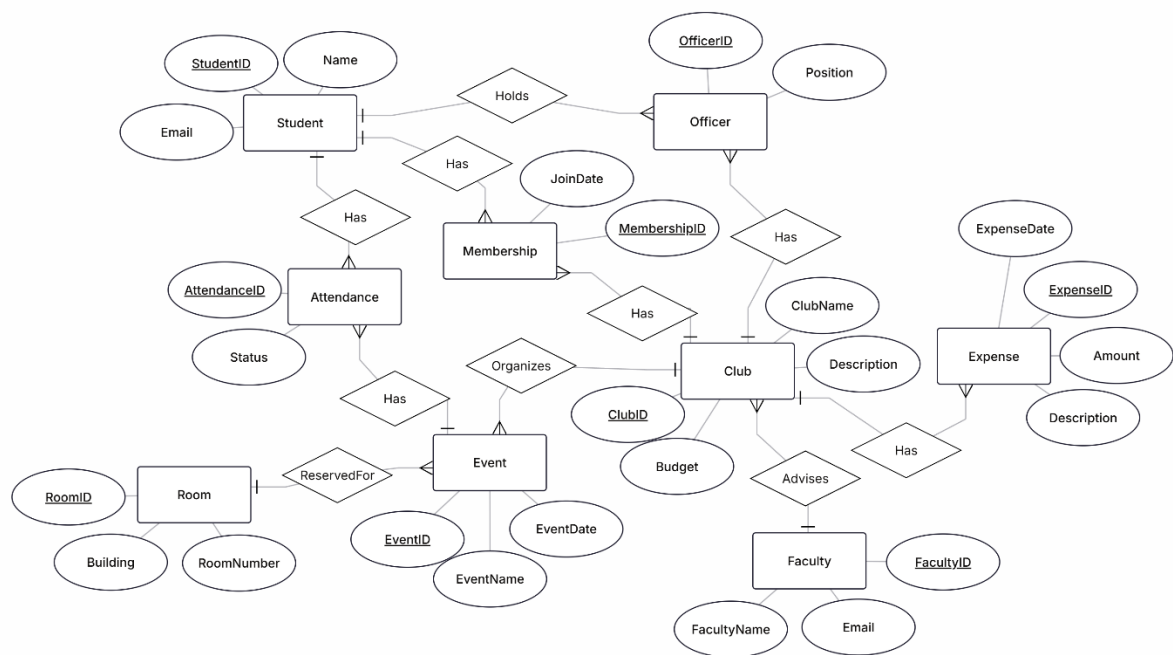
- $(\text{StudentID}, \text{CourseID}, \text{TimeSlot}, \text{Room})$

5) Potential Loss of Information

The decomposition is designed to be lossless. The original CourseSchedule information can be reconstructed by joining STUDENT, ENROLLMENT, COURSE_SECTION, COURSE, INSTRUCTOR, and ROOM through their primary-key and foreign-key relationships. Therefore, no information should be lost as long as referential integrity is maintained.

Task 5. Real-World Application

5.1. ERD



5.2. Normalized relational schema

STUDENT (StudentID PK, Name, Email)	FACULTY(FacultyID PK, Name, Email)	CLUB(ClubID PK, ClubName, Description, Budget, FacultyID FK)
MEMBERSHIP(MembershipID PK, StudentID FK, ClubID FK, JoinDate)	EVENT(EventID PK, ClubID FK, EventName, EventDate, RoomID FK)	ROOM(RoomID PK, RoomNumber, Building)
ATTENDANCE(AttendanceID PK, StudentID FK, EventID FK, Status)	OFFICER(OfficerID PK, StudentID FK, ClubID FK, Position)	EXPENSE(ExpenseID PK, ClubID FK, Amount, Description, ExpenseDate)

5.3. Design Decision: Club membership could be represented as a direct many-to-many relationship between Student and Club. However, I created a separate Membership entity because it allows additional information, such as JoinDate, to be stored for each membership. This design is also easier to extend in the future.

5.4. Three example queries

- Find all students who are members of a specific club.
- List all events organized by clubs with their room reservations.
- Find all officers of a specific club and show their positions.